DAY 09 · THEORY HANDOUT

CNN: Learn Patterns in Images

Convolutional Neural Network: a neural network that learns image patterns to help choose a class.

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

1. A photo becomes grids of numbers

A **pixel** is one tiny square in a picture. Its numbers describe brightness or color.

Photo



Grayscale: one grid

20	90	180
45	110	210
70	160	255

One brightness channel

RGB: three grids

Red

180	70	20
140	60	15
120	30	10

Green

100	160	20
90	130	30
70	110	40

Blue

30	80	190
40	100	160
70	90	140

Three color channels

A **matrix** is a grid with rows and columns. A **channel** holds one part of the image: brightness, red, green, or blue.

We resize RGB photos to **180 x 180 pixels** so inputs have the same size. **Rescaling** divides values by **255** to get numbers from 0 to 1.

> The small grids above use made-up values. At the start, we keep the rows, columns, and channels.

2. Patch, filter, and kernel

A **patch** is the small part of the image covered by the filter at one position.

Input: 5 x 5 grid

3	1	0	0	0
0	2	1	0	0
0	0	3	1	0
0	0	0	4	1
0	0	0	0	3

Patch: from the image

3	1	0
0	2	1
0	0	3

The orange window covers these 3 x 3 pixels.

Filter = kernel

1	0	0
0	1	0
0	0	1

A 3 x 3 grid of weights belongs to the model.

A **filter**, also called a **kernel**, is a small grid of weights. A **weight** is a number learned in **training**: learning from photos with known answers.

Why a small window? Nearby pixels can form a line, edge, or texture. We look for these small patterns across the image.

> The patch contains image values. The kernel contains model weights. Our small example uses fixed weights.

3. Convolution makes a feature map

Convolution means moving the same filter across the input and doing this small calculation.

1. Place the filter

3	1	0
0	2	1
0	0	3

x

1	0	0
0	1	0
0	0	1

2. Multiply and add

3 x 1 + 2 x 1 + 3 x 1

= 8

The other weights are zero, so their products add nothing.

3. Move and repeat

8	3	0
0	9	3
0	0	10

Feature map: 3 x 3

One result for each position

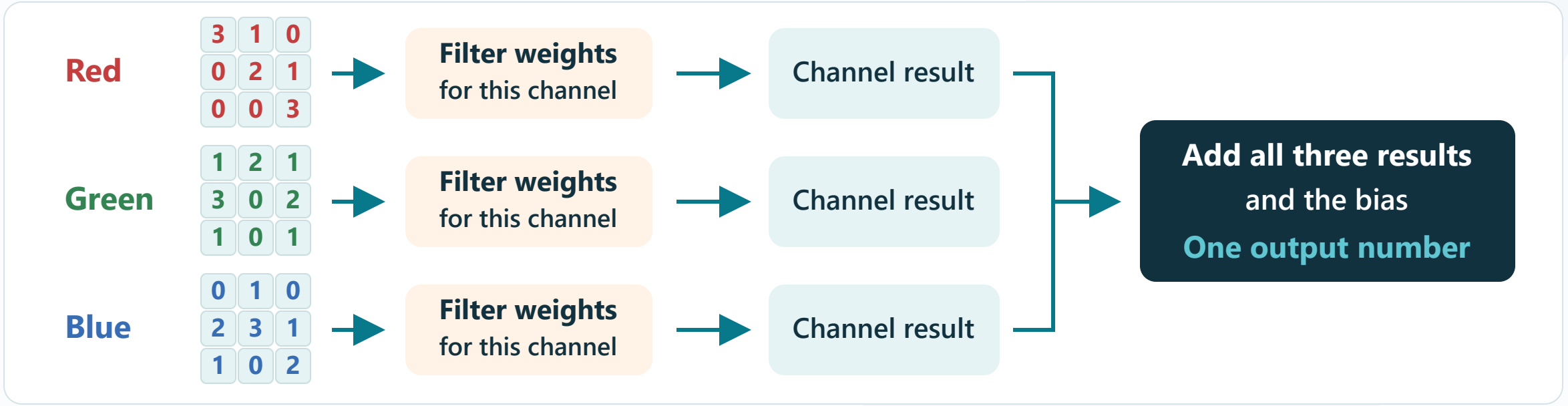
A **feature map** is the grid of results. A **feature** is a useful pattern, such as an edge. Different weights give different responses.

The same weights are reused at every position. Our example adds no bias. A **bias** is a learned number added to shift the result.

> This map has pattern responses, not cat or dog labels. A stronger response alone does not identify an animal.

4. An RGB filter uses all three channels

At each position, one filter looks at the red, green, and blue patches together.



One RGB filter has weights for **all three channels**. It adds their results to make one number at that position. Sliding it makes **one map**.

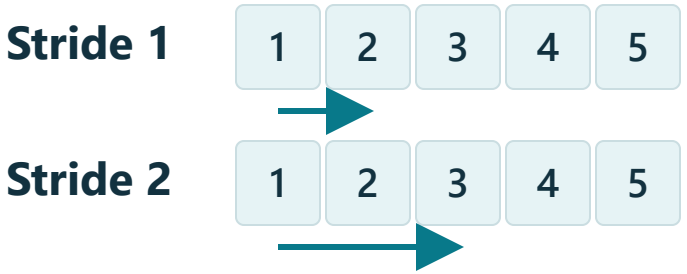
Many filters make many maps. For example, 32 filters make 32 feature maps. Later filters read the earlier maps as their input channels.

> An RGB image has three input grids. Each filter looks across all three together and makes one feature map.

5. Stride and padding control the window

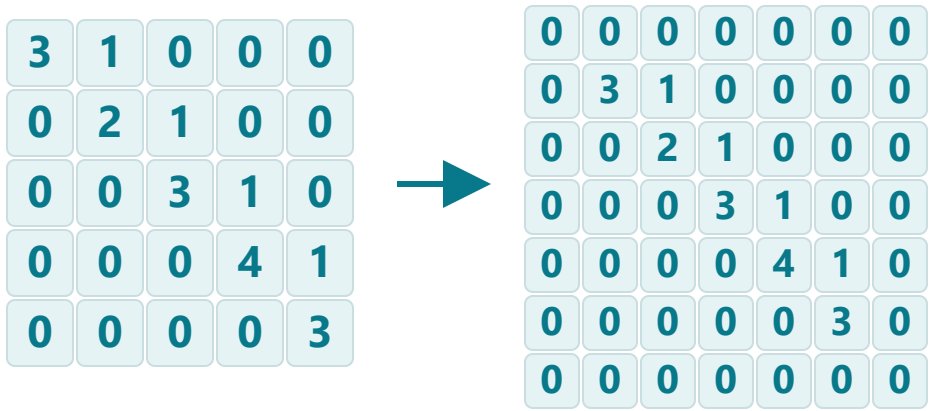
Stride is the move distance. **Padding** adds a border, often filled with zeros.

Stride = how far we move



A bigger stride checks fewer positions.

Padding = an extra border



5 x 5 input

7 x 7 with zero border

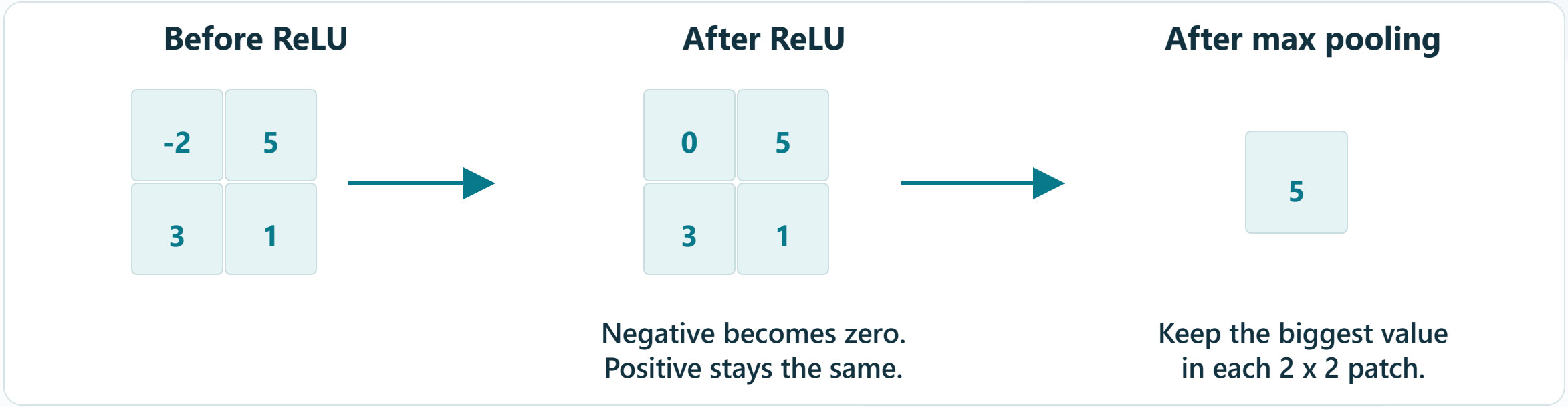
Why stride? Stride 1 checks nearby positions. Stride 2 skips farther, giving a smaller map and fewer calculations.

Why padding? The extra border lets the filter cover positions near the edges. It can also stop the map from shrinking.

With a 3 x 3 filter and stride 1: a 5 x 5 grid gives a 3 x 3 map without padding, or a 5 x 5 map with one zero border.

6. ReLU and pooling change the map

ReLU is an activation function: a rule that changes the numbers from the previous step.



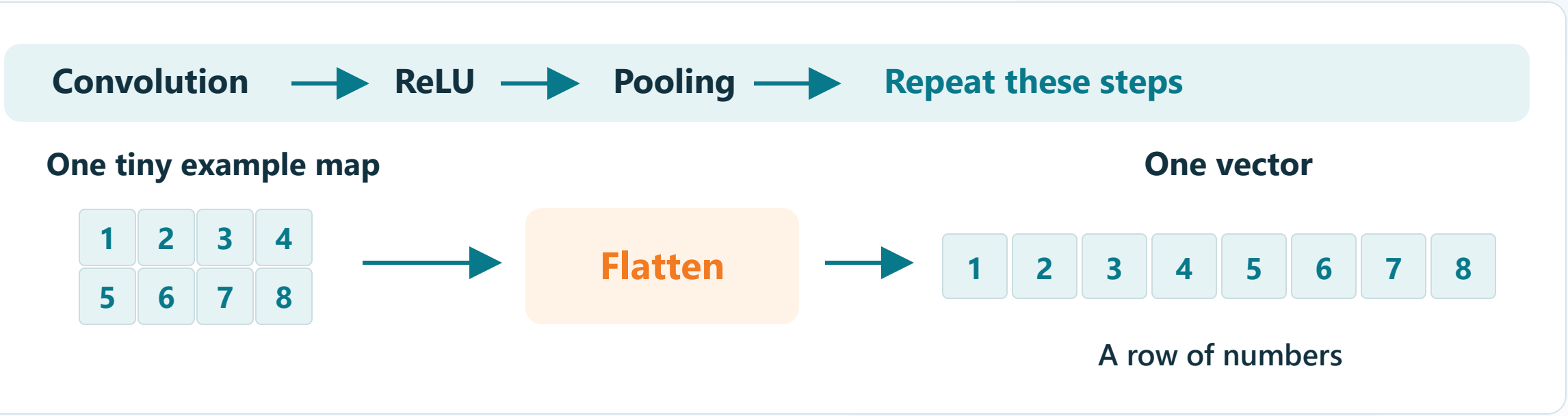
ReLU means **Rectified Linear Unit**. This simple rule helps the network learn patterns that straight-line rules alone cannot describe.

Max pooling keeps the biggest value in each small patch. It makes maps smaller, so the next steps do less work. It has no weights to learn.

> ReLU changes values. Pooling reduces the size. Neither step gives the final cat-or-dog answer.

7. Repeat, then Flatten

A **layer** is one processing step. Later layers can combine the earlier pattern responses.



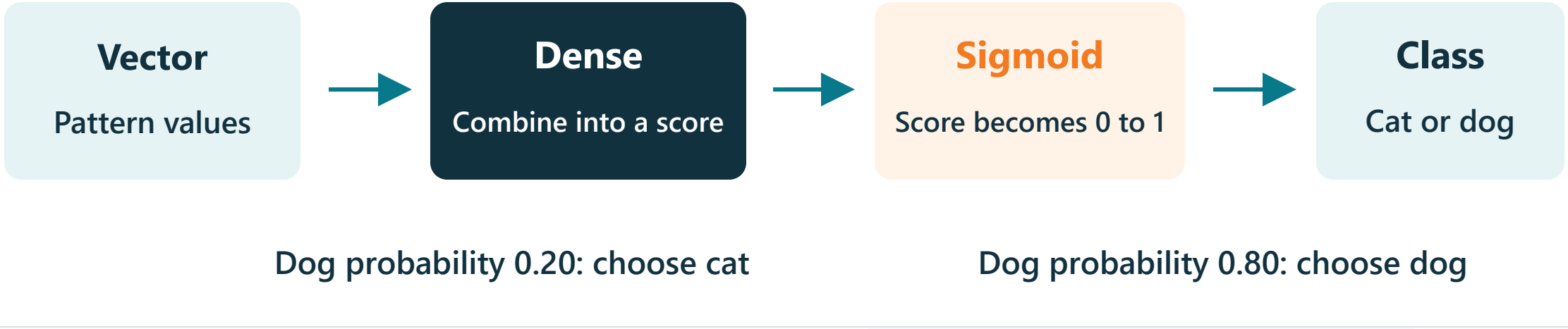
Early layers can respond to simple edges or textures. Later layers can combine these into larger patterns. Learned filters do not come with names like "ear".

Flatten rearranges the maps into a **vector**, a one-dimensional row of numbers. It keeps the values so the next layer can combine them.

> This is where our model makes one vector. Pooling shrinks maps; Flatten changes how their values are arranged.

8. Dense and sigmoid choose a class

Classification means choosing a category. Our two categories, or **classes**, are cat and dog.



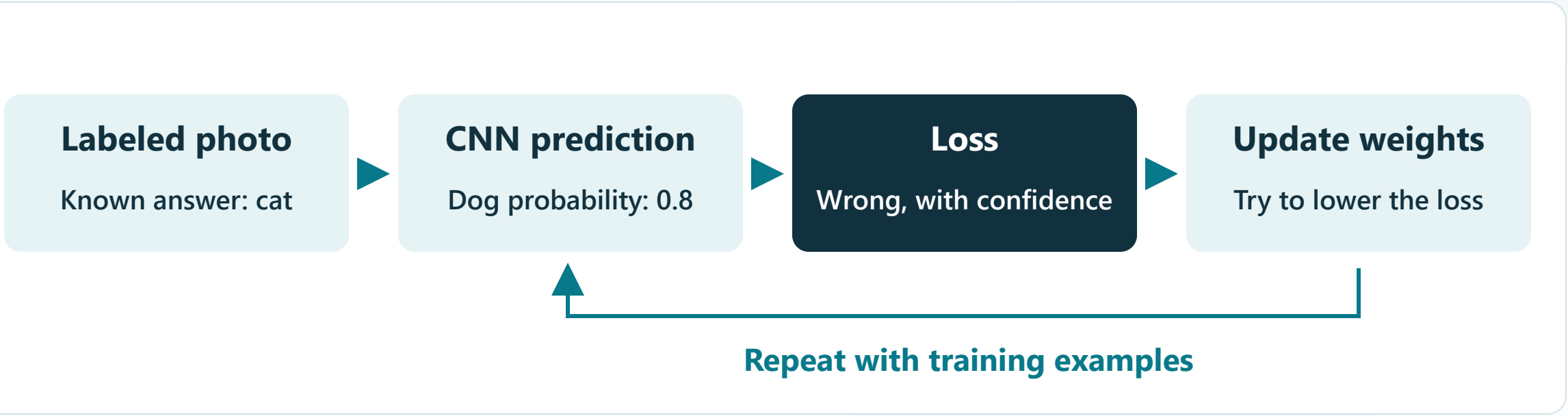
A **Dense layer** combines the input values using learned weights and a bias. Our final Dense layer makes one score.

Sigmoid turns that score into a number from 0 to 1. We use it as the model's **probability** for dog: its estimated chance that this is a dog.

Our labels are cat = 0 and dog = 1. Dog probability below 0.5 gives cat; 0.5 or above gives dog. An estimate can be wrong.

9. Training: how the weights improve

A **label** is the known answer. **Loss** is a number that measures how far the prediction is from that answer.



Backpropagation works backward through the layers to find how weights affect the loss. The **optimizer** uses this information to change the weights.

Our loss is **binary cross-entropy**, used for two classes. It gives a bigger penalty for a confident wrong answer. Lower loss is the training aim.

> Prediction follows the same image-to-class path using the learned weights. It does not update those weights.

10. Our project: test a new image

Resize the RGB photo, rescale once, then use the learned CNN to get a cat-or-dog prediction.



New cat: predicted dog

Dog probability: 0.567



New dog: predicted dog

Dog probability: 0.903

Check unseen photos

Test accuracy: 68.6%

1,372 correct out of 2,000

Accuracy = fraction of test labels predicted correctly

Training set: photos used to learn weights. **Validation set:** separate photos used to choose the saved model. **Test set:** unseen photos for the final score.

The two photos above were outside all three sets. A new photo can still be wrong. The supplied horse photo is neither class; the model still forces cat or dog.

> The model learns cat and dog only. A high dog probability does not prove that an unfamiliar object is a dog.

DAY 09 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie

Laxmi Kant Tiwari · kgptalkie.com